ISRO

Bharatiya Antariksh

Hackathon 2025

Urban Heat Stress Hotspot Detection

using Geospatial AI/ML

5-Day Implementation Roadmap for CS Student Teams

A complete guide: datasets · methodology · architecture · code · presentation

Python | Google Earth Engine | Streamlit | Scikit-learn | Folium | GeoPandas

Table of Contents

1.	Introduction & Problem Statement	3
2.	Quick-Start Checklist	4
3.	Day 1 — Setup & Data Acquisition	5
4.	Day 2 — Preprocessing & LST Calculation	8
5.	Day 3 — Hotspot Detection & AI/ML Model	11
6.	Day 4 — Dashboard & Visualization	15
7.	Day 5 — Integration, Testing & Demo	18
8.	Datasets Reference	20
9.	Methodology Deep Dive	23
10.	Technology Stack	27
11.	System Architecture	29
12.	Development Setup & Folder Structure	32
13.	Final Demo Plan	35
14.	PPT Structure Guide	37
15.	MVP vs Advanced Features	39

1. Introduction & Problem Statement

Urban Heat Islands (UHI) are metropolitan areas significantly warmer than surrounding rural areas due to human activities and land-use changes. The Bharatiya Antariksh Hackathon challenges you to build an AI/ML-powered system using satellite data to **detect, map, and predict heat stress hotspots** in Indian cities — directly impacting public health and urban planning.

Challenge	Urban Heat Stress Hotspot Detection using Geospatial AI/ML
Organizer	ISRO — Indian Space Research Organisation
Team	CS students with limited GIS experience
Timeline	5 days (Hackathon format)
Primary Data	Landsat 8/9, Sentinel-2, MODIS LST, Census population
Output	Interactive dashboard with real-time hotspot detection & cooling recs

Why This Matters

- Heat stress causes 1500+ deaths annually in India (WHO data)
- Urban temperatures in Indian cities are rising 0.5°C per decade
- ISRO satellite data (Resourcesat, Cartosat) is freely available
- AI/ML can identify vulnerable populations and suggest interventions
- Your solution can directly feed into Smart City Mission planning

2. Quick-Start Checklist (Do This First!)

■ **Note:** Complete all items in this checklist on Day 1 morning before starting development.

#	Task	Owner	Time
1	Create Google Earth Engine account at earthengine.google.com	All	15 min
2	Install Python 3.10+ and Anaconda/Miniconda	All	20 min
3	Clone starter GitHub repository (create new if not available)	Lead Dev	10 min
4	Install all required Python libraries (see Section 12)	All	30 min
5	Download Landsat 8 scene for your chosen city from USGS EarthExplorer	Data Lead	45 min
6	Download Sentinel-2 data from Copernicus Open Access Hub	Data Lead	30 min
7	Download city boundary shapefile from GADM or Bhuvan	Data Lead	20 min
8	Download population data from Census India 2011 website	Data Lead	20 min
9	Authenticate GEE in Python: earthengine authenticate	ML Lead	10 min
10	Set up project folder structure (see Section 12)	Lead Dev	15 min
11	Assign roles: Data Lead, ML Lead, Visualization Lead, Documentation Lead	All	10 min
12	Decide target city (Delhi, Mumbai, Bengaluru, Chennai or Hyderabad recommended)	All	15 min

✓ **Tip:** Use Delhi or Mumbai — they have excellent open data availability and clear heat island patterns.

DAY 1 Environment Setup & Data Acquisition

Objective: Set up the full development environment, acquire all datasets, and validate data quality.

Tasks — Morning (4h)

- Install Python, Anaconda, VS Code, QGIS (for visual verification)
- Create conda environment: conda create -n heat_ai python=3.10
- Install all libraries (see requirements.txt in Section 12)
- Create Google Earth Engine account and authenticate
- Set up GitHub repository with proper folder structure

Tasks — Afternoon (4h)

- Download Landsat 8/9 Band 4, 5, 10, 11 from USGS EarthExplorer
- Download Sentinel-2 Level-2A product from Copernicus Hub
- Download MODIS MOD11A2 LST 8-day composite from LP DAAC
- Download city boundary shapefile from GADM.org
- Download population density raster from WorldPop Hub
- Download OpenStreetMap data for roads, parks, water bodies

Deliverables

- ■ Working conda environment with all libraries installed
- ■ Authenticated Google Earth Engine access
- ■ GitHub repo with initial folder structure committed
- ■ Raw datasets downloaded and organized in /data/raw/
- ■ Data inventory spreadsheet (filename, date, resolution, source)

Team Responsibilities

Lead Dev	GitHub setup, environment configuration, folder structure
Data Lead	Downloading and organizing all datasets
ML Lead	GEE authentication, initial GEE script testing
Docs Lead	Data inventory spreadsheet, README initial draft

Success Criteria

- ✓ All 4 team members can run: python -c "import ee; ee.Initialize()" without error
- ✓ At least 1 complete Landsat scene covers the study city
- ✓ Folder structure matches the template in Section 12

DAY 2 Data Preprocessing & LST Calculation

Objective: Preprocess all raw data, calculate Land Surface Temperature, and map Urban Heat Islands.

Tasks — Morning (4h)

- Cloud masking on Landsat using QA_PIXEL band
- Radiometric calibration: convert DN to Top-of-Atmosphere reflectance
- Atmospheric correction using DOS1 or 6S method
- Calculate NDVI from Bands 4 & 5: NDVI = (B5-B4)/(B5+B4)
- Calculate NDWI (water index) and NDBI (built-up index)
- Clip all rasters to study area boundary

Tasks — Afternoon (4h)

- Convert Landsat Band 10 to Brightness Temperature (BT)
- Calculate Land Surface Emissivity (LSE) from NDVI
- Apply Radiative Transfer Equation to derive LST in Celsius
- Calculate UHI index: UHI = (LST_pixel - LST_mean) / LST_std
- Reclassify LST into 5 heat zones (Very Low to Extreme)
- Export processed rasters to /data/processed/

Deliverables

- ■ LST raster (.tif) for study city at 30m resolution
- ■ NDVI, NDWI, NDBI rasters clipped to city boundary
- ■ UHI index raster with 5-class heat zone classification
- ■ Statistics CSV: mean/max/min LST by ward or district
- ■ Quick visualization: matplotlib heat map for verification

Team Responsibilities

Lead Dev	Raster processing pipeline scripts, data I/O
Data Lead	Quality checking outputs, statistics generation
ML Lead	Feature engineering from rasters (extract pixel values)
Docs Lead	Document all formulas used, update README

Success Criteria

- ✓ LST values are realistic: 25–55°C range for Indian summer cities
- ✓ NDVI range: -0.1 to 0.8 (negative for water, high for dense vegetation)

– ✓ No more than 5% cloud contamination in processed LST raster

DAY 3 Hotspot Detection & AI/ML Model

Objective: Detect and cluster heat hotspots, build and train the AI/ML prediction model.

Tasks — Morning (4h)

- Getis-Ord Gi* statistic for spatial hotspot detection using PySAL
- DBSCAN clustering to group hotspot pixels into zones
- K-Means clustering as baseline comparison
- Calculate hotspot severity scores (combine LST + population + NDVI)
- Identify top-20 most critical hotspot zones with coordinates

Tasks — Afternoon (5h)

- Prepare ML feature matrix: LST, NDVI, NDBI, NDWI, population density, distance to water, elevation
- Create training labels: heat stress index (composite score)
- Train Random Forest Regressor for LST prediction
- Train Gradient Boosting for hotspot severity classification
- Cross-validation: 80/20 train-test split with k-fold
- Evaluate: RMSE, MAE, R^2 score, confusion matrix
- SHAP values for feature importance visualization

Deliverables

- ■ Hotspot detection results: GeoJSON with top-20 hotspot polygons
- ■ Trained Random Forest model saved as .pkl file
- ■ Model evaluation report: accuracy metrics and confusion matrix
- ■ Feature importance chart (SHAP or sklearn built-in)
- ■ Hotspot severity ranking CSV with coordinates and scores

Team Responsibilities

Lead Dev	ML pipeline code, model training scripts
Data Lead	Feature matrix preparation, training data quality checks
ML Lead	Model architecture, hyperparameter tuning, evaluation
Docs Lead	Document model methodology, record experiment results

Success Criteria

- ✓ Random Forest achieves $R^2 > 0.75$ on test set
- ✓ Hotspot clusters are spatially coherent (visual sanity check)

– ✓ Model training completes within 30 minutes on laptop

DAY 4 Dashboard Development & Visualization

Objective: Build an interactive Streamlit dashboard with Folium maps and cooling recommendations.

Tasks — Morning (4h)

- Design dashboard layout: sidebar + 3-tab main area
- Tab 1: City Overview — interactive Folium heat map with LST layer
- Tab 2: Hotspot Analysis — clustered hotspots with severity indicators
- Tab 3: Cooling Recommendations — AI-powered interventions
- Add date picker for temporal analysis (if multi-date data available)

Tasks — Afternoon (4h)

- Integrate ML model predictions into dashboard (real-time scoring)
- Add population overlay to show vulnerable area counts
- Create bar charts: LST by land-use type, time-series trend
- Implement cooling recommendation engine (rule-based + ML)
- Add download buttons: CSV export, PDF report generation
- Deploy locally and test on all team members' machines

Deliverables

- ■ Working Streamlit app (app.py) running on localhost:8501
- ■ Interactive Folium map with LST, hotspot, and population layers
- ■ Cooling recommendations panel with location-specific suggestions
- ■ Download functionality for hotspot reports
- ■ Screenshots of all dashboard screens for PPT

Team Responsibilities

Lead Dev	Streamlit app structure, tab logic, deployment
Data Lead	Data loading functions, caching, performance optimization
ML Lead	ML inference integration, real-time prediction endpoint
Docs Lead	UI text, help tooltips, taking screenshots for PPT

Success Criteria

- – ✓ Dashboard loads in under 10 seconds
- – ✓ All 3 tabs render correctly with real data
- – ✓ Folium map shows heat zones with correct color coding

DAY 5 Integration, Testing, PPT & Demo Rehearsal

Objective: Final integration, bug fixing, PPT preparation, and demo rehearsal.

Tasks — Morning (4h)

- End-to-end integration test: raw data → processed → ML → dashboard
- Fix critical bugs identified during integration testing
- Performance optimization: add @st.cache_data for heavy computations
- Prepare final demo dataset (pre-processed, fast loading)
- Write unit tests for critical functions (LST calculation, hotspot detection)

Tasks — Afternoon (4h)

- Build PowerPoint presentation (follow structure in Section 14)
- Prepare 5-minute demo script with talking points
- Practice demo flow: narration + live dashboard + key findings
- Prepare Q&A; answers for common judge questions
- Final GitHub commit: clean code, comprehensive README
- Optional: deploy to Streamlit Cloud for live demo URL

Deliverables

- ■ Fully functional integrated system (data → dashboard)
- ■ Final PowerPoint presentation (10-12 slides)
- ■ Demo video (2-3 minutes) as backup in case of live issues
- ■ Clean GitHub repository with README and requirements.txt
- ■ Q&A; preparation document with anticipated judge questions

Team Responsibilities

Lead Dev	Integration testing, bug fixes, GitHub cleanup
Data Lead	Prepare final demo data, verify all statistics
ML Lead	Final model validation, prepare model metrics slide
Docs Lead	PPT creation, demo script, README finalization

Success Criteria

- ✓ Full demo runs end-to-end without errors
- ✓ PPT is complete with real results (not placeholder data)
- ✓ GitHub repo is presentable with star-worthy README

8. Datasets Reference

Landsat 8/9 OLI-TIRS

Primary satellite for Land Surface Temperature calculation

Bands/Layers	Band 4 (Red), Band 5 (NIR), Band 10 (TIRS-1), Band 11 (TIRS-2)
Resolution	30m optical, 100m thermal (resampled to 30m)
Download From	USGS EarthExplorer — earthexplorer.usgs.gov
Format	GeoTIFF (.TIF)
Cost	Yes — completely free

Download Steps:

- Go to earthexplorer.usgs.gov, create free account
- Search by coordinates of your city center
- Filter: Date Range = May-June 2023, Cloud Cover < 10%
- Dataset = Landsat Collection 2 Level-1
- Download the full scene (~1 GB per scene)

Sentinel-2 MSI Level-2A

High-resolution optical imagery for land use classification

Bands/Layers	B2 Blue, B3 Green, B4 Red, B8 NIR, B11 SWIR
Resolution	10m (B2/3/4/8), 20m (B11/12)
Download From	Copernicus Open Access Hub — scihub.copernicus.eu
Format	JPEG2000 (.jp2)
Cost	Yes — completely free

Download Steps:

- Register at scihub.copernicus.eu
- Search by city name or draw bounding box
- Filter: Sentinel-2, Level-2A, Cloud Cover < 5%
- Select summer scene (April-June 2023)
- Download entire product zip (~600 MB)

MODIS MOD11A2 LST

8-day average LST composite for temporal analysis

Bands/Layers	LST_Day_1km, QC_Day, LST_Night_1km
Resolution	1km
Download From	LP DAAC — lpdaac.usgs.gov or via GEE
Format	HDF4 or via GEE as ImageCollection

Cost	Yes

Download Steps:

- In Google Earth Engine console, use:
- ee.ImageCollection("MODIS/061/MOD11A2")
- Filter by date and city boundary
- Select "LST_Day_1km" band
- Scale by 0.02 and subtract 273.15 for Celsius

WorldPop Population Density

100m resolution gridded population estimates

Bands/Layers	Population count per pixel
Resolution	100m
Download From	WorldPop Hub — hub.worldpop.org
Format	GeoTIFF
Cost	Yes

Download Steps:

- Go to hub.worldpop.org
- Select India > Population > Unconstrained individual countries
- Download IND_ppp_2020_1km_Aggregated.tif
- Alternatively use GEE: ee.Image("WorldPop/GP/100m/pop/IND_2020")

OpenStreetMap (OSM)

Roads, parks, water bodies, building footprints

Bands/Layers	Vector: points, lines, polygons
Resolution	Vector (no fixed resolution)
Download From	Geofabrik — download.geofabrik.de/asia/india
Format	Shapefile or GeoJSON
Cost	Yes — ODbL license

Download Steps:

- Go to download.geofabrik.de/asia/india
- Download india-latest-free.shp.zip (~600 MB)
- Or use osmnx Python library for specific city:
- import osmnx as ox
- G = ox.graph_from_place('Delhi, India')

GADM Administrative Boundaries

City/district/ward boundary shapefiles for India

Bands/Layers	Vector polygons
Resolution	Vector
Download From	gadm.org
Format	Shapefile, GeoJSON, GeoPackage
Cost	Yes — for non-commercial use

Download Steps:

- Go to gadm.org/download_country.html
- Select India, choose level 2 (districts) or level 3 (subdistricts)
- Download as Shapefile format
- Open in QGIS to verify and select your city boundary
- Export city boundary as separate shapefile

9. Methodology Deep Dive

9.1 Land Surface Temperature (LST) Calculation

LST is derived from Landsat Band 10 thermal infrared data in 5 steps:

Step 1: DN to Spectral Radiance

```
L = ML × Qcal + AL
ML = RADIANCE_MULT_BAND_10 (from MTL file)
AL = RADIANCE_ADD_BAND_10
Qcal = raw pixel digital number
```

✓ **Tip:** Output: Radiance in W/(m²·sr·µm)

Step 2: Radiance to Brightness Temperature (BT)

```
BT = K2 / ln(K1/L + 1) - 273.15
K1 = 774.8853 (constant for Band 10)
K2 = 1321.0789 (constant for Band 10)
```

✓ **Tip:** Output: Brightness Temperature in Celsius

Step 3: Calculate NDVI

```
NDVI = (NIR - Red) / (NIR + Red)
= (Band5 - Band4) / (Band5 + Band4)
```

✓ **Tip:** Range: -1 to +1. Vegetation > 0.2

Step 4: Calculate Land Surface Emissivity (LSE)

```
Pv = ((NDVI - NDVIs) / (NDVIv - NDVIs))^2
NDVIs = 0.2 (bare soil NDVI)
NDVIv = 0.5 (full vegetation NDVI)
LSE = 0.004 × Pv + 0.986
```

✓ **Tip:** Emissivity accounts for surface thermal properties

Step 5: Final LST Calculation

```
LST = BT / (1 + (lambda × BT / rho) × ln(LSE))
lambda = 10.895 µm (Band 10 wavelength)
rho = 14388 µm·K (h × c / sigma)
```

✓ **Tip:** Output: True surface temperature in Celsius

9.2 Urban Heat Island (UHI) Index

The UHI index normalizes LST to show relative heat compared to the city average:

```
UHI_index = (LST_pixel - LST_mean) / LST_std

# Classification thresholds:
# UHI < -1.5 → Very Low Heat Zone
```

```
# UHI -1.5 to -0.5 → Low Heat Zone
# UHI -0.5 to +0.5 → Moderate Heat Zone
# UHI +0.5 to +1.5 → High Heat Zone
# UHI > +1.5 → Extreme Heat Zone (HOTSPOT)
```

9.3 Hotspot Detection — Getis-Ord Gi* Statistic

The Gi* statistic identifies statistically significant spatial clusters of high values. A high positive Z-score means a hotspot (cluster of high LST values). A high negative Z-score means a coldspot.

```
from pysal.explore import esda
from pysal.lib import weights
import libpysal

# Create spatial weights matrix
w = weights.Queen.from_dataframe(gdf)
w.transform = "r" # Row-standardize

# Calculate Getis-Ord Gi*
gi = esda.G_Local(gdf["LST"], w, transform="B", permutations=999)

# Significant hotspots: z_sim > 1.96 (p < 0.05)
gdf["hotspot"] = gi.z_sim > 1.96
gdf["coldspot"] = gi.z_sim < -1.96
```

9.4 DBSCAN Hotspot Clustering

```
from sklearn.cluster import DBSCAN
import numpy as np

# Extract hotspot pixel coordinates
hotspot_coords = gdf[gdf["hotspot"]== True][["lon","lat"]].values

# DBSCAN: eps=0.005 degrees (~500m), min_samples=50 pixels
db = DBSCAN(eps=0.005, min_samples=50, algorithm="ball_tree",
metric="haversine").fit(np.radians(hotspot_coords))

gdf.loc[gdf["hotspot"]==True, "cluster_id"] = db.labels_

# Noise points (label=-1) are isolated hotspot pixels
n_clusters = len(set(db.labels_)) - (1 if -1 in db.labels_ else 0)
print(f"Found {n_clusters} hotspot zones")
```

9.5 AI/ML Prediction Model

Build a Random Forest model to predict heat stress scores for new scenarios:

```
from sklearn.ensemble import RandomForestRegressor, GradientBoostingClassifier
from sklearn.model_selection import train_test_split, cross_val_score
from sklearn.metrics import mean_squared_error, r2_score
```

```
import joblib

# Feature matrix
features = ["LST","NDVI","NDBI","NDWI","pop_density",
"dist_water","elevation","imperv_fraction"]
X = gdf[features].values
y = gdf["heat_stress_index"].values

# Split
X_train, X_test, y_train, y_test = train_test_split(
X, y, test_size=0.2, random_state=42)

# Train
rf = RandomForestRegressor(n_estimators=200, max_depth=12,
min_samples_leaf=5, n_jobs=-1, random_state=42)
rf.fit(X_train, y_train)

# Evaluate
y_pred = rf.predict(X_test)
print(f"R²: {r2_score(y_test, y_pred):.3f}")
print(f"RMSE: {mean_squared_error(y_test, y_pred, squared=False):.3f}")

# Save model
joblib.dump(rf, "models/heat_stress_rf.pkl")
```

9.6 Validation Approach

- Cross-validation: 5-fold cross validation with R^2, RMSE, MAE
- Ground truth: Compare LST predictions against MODIS LST for known dates
- Spatial validation: Overlay predictions on Google Earth imagery — do hotspots align with industrial/dense areas?
- Statistical: Residual plots, Q-Q plots for model diagnostics
- Expert validation: Compare identified hotspots with newspaper reports of heat wave deaths

10. Technology Stack

Library	Category	Use in Project	Documentation
Python 3.10+	Core language	All data processing, ML, visualization	python.org
Google Earth Engine	Satellite data platform	Access and process petabytes of satellite data	earthengine.google.com
GeoPandas	Geospatial dataframes	Vector data: shapefiles, GeoJSON, spatial joins	geopandas.org
Rasterio	Raster processing	Read/write GeoTIFF, raster math, reprojection	rasterio.readthedocs.io
NumPy + SciPy	Numerical computing	Array math, statistical functions, LST formulas	numpy.org
Scikit-learn	ML framework	Random Forest, clustering, cross-validation, metrics	scikit-learn.org
Folium	Interactive maps	Leaflet.js-based maps in Streamlit and HTML	python-visualization.github.io/folium
Streamlit	Dashboard framework	Web app with zero frontend code required	streamlit.io
Matplotlib + Seaborn	Static charts	Heat maps, histograms, feature importance plots	matplotlib.org
PySAL	Spatial statistics	Getis-Ord Gi* hotspot detection	pysal.org
GDAL/OGR	Geospatial engine	Backend for rasterio and geopandas	gdal.org
Pandas	Data manipulation	CSVs, statistics, feature engineering	pandas.pydata.org

11. System Architecture

Data Pipeline

1. Raw Satellite Data (Landsat, Sentinel, MODIS) → USGS/Copernicus/GEE
2. Ancillary Data (Population, Boundaries, OSM) → WorldPop/GADM/Geofabrik
3. Data Ingestion Module (Python: rasterio, geopandas) → /data/raw/
4. Cloud Masking & QA → Reproject to WGS84/UTM → Clip to AOI
5. Radiometric Correction → /data/processed/ → PostGIS or GeoTIFF

Processing Pipeline

1. Band Math Module: NDVI, NDWI, NDBI, BT calculations
2. LST Engine: 5-step radiative transfer equation implementation
3. UHI Calculator: normalize LST → 5-class heat zone raster
4. Feature Extractor: sample pixel values → structured DataFrame
5. Output: features.csv, lst.tif, uhi.tif, zones.geojson

AI/ML Pipeline

1. Feature Engineering: scaling, encoding, train/test split
2. Hotspot Detection: Getis-Ord Gi* → DBSCAN clustering
3. ML Training: Random Forest Regressor + Gradient Boosting Classifier
4. Model Evaluation: cross-validation, SHAP values, metrics report
5. Model Registry: joblib pkl files versioned in /models/ directory

Visualization Pipeline

1. Folium Map Builder: LST layer, hotspot overlay, population heatmap
2. Chart Generator: matplotlib bar charts, time-series, scatter plots
3. Streamlit App: multi-tab dashboard with sidebar controls
4. Export Engine: CSV download, PNG chart export, PDF summary report
5. Optional: Streamlit Cloud deployment for shareable URL

Recommendation Engine

1. Rule Engine: IF hotspot_severity > 0.8 AND population > 5000 THEN priority=CRITICAL
2. Intervention Database: 15 pre-defined cooling interventions with costs and impact scores

3. Prioritization Algorithm: score interventions by (impact × population) / cost
4. Output: Location-specific ranked list of cooling recommendations
5. Future: Reinforcement learning for adaptive recommendation optimization

12. Development Setup & Folder Structure

12.1 Folder Structure

```
heat_stress_detection/
■■■ README.md
■■■ requirements.txt
■■■ .gitignore
■■■ app.py # Main Streamlit app
■■■ data/
■ ■■■ raw/ # Downloaded original files (git-ignored)
■ ■ ■■■ landsat/
■ ■ ■■■ sentinel/
■ ■ ■■■ ancillary/
■ ■■■ processed/ # Processed rasters & vectors
■ ■ ■■■ lst_delhi.tif
■ ■ ■■■ uhi_zones.geojson
■ ■ ■■■ features.csv
■ ■■■ demo/ # Pre-loaded demo data for fast dashboard
■■■ notebooks/ # Jupyter exploration notebooks
■ ■■■ 01_data_explore.ipynb
■ ■■■ 02_lst_calculation.ipynb
■ ■■■ 03_ml_model.ipynb
■■■ src/ # Python modules
■ ■■■ __init__.py
■ ■■■ data_loader.py # Data loading functions
■ ■■■ preprocessing.py # Cloud masking, reprojection
■ ■■■ lst_calculator.py # LST formula implementation
■ ■■■ hotspot_detector.py # Gi* and DBSCAN
■ ■■■ ml_model.py # ML training and inference
■ ■■■ visualizer.py # Map and chart functions
■ ■■■ recommender.py # Cooling recommendations
■■■ models/ # Saved ML models
■ ■■■ heat_stress_rf.pkl
■■■ tests/ # Unit tests
■ ■■■ test_lst.py
■■■ docs/
■■■ methodology.md
■■■ presentation.pptx
```

12.2 Installation Commands

```
# Step 1: Create conda environment
conda create -n heat_ai python=3.10 -y
conda activate heat_ai

# Step 2: Install GDAL via conda (must be first!)
conda install -c conda-forge gdal rasterio geopandas -y
```

```
# Step 3: Install remaining packages via pip
pip install earthengine-api streamlit folium scikit-learn
pip install numpy pandas matplotlib seaborn joblib
pip install pysal libpysal esda splot
pip install osmnx shap plotly

# Step 4: Authenticate Google Earth Engine
earthengine authenticate

# Step 5: Verify installation
python -c "import ee; import rasterio; import geopandas; print('All good!')"

# Step 6: Run the Streamlit app
streamlit run app.py
```

12.3 requirements.txt

```
earthengine-api==0.1.370
streamlit==1.31.0
geopandas==0.14.3
rasterio==1.3.9
folium==0.15.1
scikit-learn==1.4.0
numpy==1.26.4
pandas==2.2.0
matplotlib==3.8.3
seaborn==0.13.2
pysal==23.7
libpysal==4.9.2
esda==2.5.1
splot==1.1.5.post1
osmnx==1.9.1
shap==0.44.1
plotly==5.19.0
joblib==1.3.2
```

12.4 GitHub Setup

```
# Initialize repository
git init
git remote add origin https://github.com/YOUR_TEAM/heat-stress-detection

# .gitignore essential entries
echo "data/raw/\ndata/processed/\n*.tif\n*.pkl\n.env" > .gitignore

# Branch strategy
main → stable, demo-ready code
dev → integration branch
feature/day1 → Day 1 work
feature/ml → ML model development
```

```
feature/ui → Dashboard development

# Commit messages
git commit -m "feat: add LST calculation module"
git commit -m "fix: cloud mask threshold adjusted to 0.1"
git commit -m "docs: add methodology documentation"
```

13. Final Demo Plan

13.1 Dashboard Screens

Screen 1: City Overview Map

- Full-page Folium interactive map of target city
- LST heat layer with orange-red color gradient
- Toggle layers: LST / NDVI / Population / Hotspots
- City statistics sidebar: max temp, mean temp, hotspot count
- Date selector for temporal comparison (if multi-date data)

Screen 2: Hotspot Analysis

- Map zoomed to top-10 critical hotspot zones
- Each hotspot labeled with severity score and population at risk
- Bar chart: top-10 hotspot zones ranked by composite risk score
- Table: zone name, area km², mean LST, population, risk category
- Click any zone to see detailed metrics

Screen 3: Land Use Analysis

- Pie chart: percentage of city area in each heat zone class
- Scatter plot: NDVI vs LST (should show inverse relationship)
- Bar chart: mean LST by land use type (commercial/residential/industrial/green)
- Statistics: correlation coefficients between LST and NDVI, NDBI, population

Screen 4: AI/ML Model Results

- Feature importance horizontal bar chart (SHAP values or RF built-in)
- Predicted vs Actual LST scatter plot with R^2 displayed
- Model metrics card: R^2, RMSE, MAE, training time
- Future scenario slider: "What if green cover increases by 10%?"

Screen 5: Cooling Recommendations

- Top-5 priority intervention recommendations for the city
- Each recommendation: location, intervention type, estimated temperature reduction, cost tier
- Types: Urban Greening, Cool Roofs, Water Features, Shade Structures, Reflective Pavements
- Impact map: highlight areas where interventions would have maximum effect

13.2 Demo Script (5-minute flow)

Time	Section	Talking Points
0:00 – 0:30	Problem Hook	Open with a striking fact: "Delhi experienced 47°C in May 2024. 12 people died from heat stroke in a single day. We're here to help prevent this."
0:30 – 1:30	City Overview	Switch to dashboard. Show the LST heat map. "The orange-red areas — that's where surface temperatures exceed 45°C. These are not random — they follow industrial zones and dense concrete areas."
1:30 – 2:30	Hotspot Zones	Navigate to Hotspot Analysis tab. "Our AI has identified 18 critical zones. The top 3 are home to 1.2 million people. This is Shaheen Bagh — 47.3°C mean surface temp, 85,000 residents."
2:30 – 3:30	AI/ML Results	Show Model Results tab. "Our Random Forest model achieves R^2=0.82. The top predictors are imperviousness fraction, distance to water bodies, and population density — confirming scientific literature."
3:30 – 4:30	Recommendations	Show Cooling Recommendations. "For Zone 3, we recommend prioritizing tree plantation on MG Road — our model predicts a 2.1°C reduction. For Zone 7, cool roof installation on 30% of buildings reduces LST by 1.8°C."
4:30 – 5:00	Impact & Close	Show impact slide. "This tool helps city planners identify exactly where to invest in cooling infrastructure. It runs on freely available satellite data and open-source tools — any Indian municipality can use it."

14. PowerPoint Presentation Structure

Slide 1: Title

- Title: "HeatSense: AI-Powered Urban Heat Stress Detection"
- Subtitle: Bharatiya Antariksh Hackathon 2025
- Team name and members
- ISRO logo, Streamlit logo, Python logo
- Background: satellite thermal image of your city (dramatic red/orange)

Slide 2: Problem Statement

- Striking statistic headline (47°C, deaths, economic losses)
- 3-column infographic: Scale of Problem | Current Gap | Our Opportunity
- Map showing India heat wave zones 2023-2024
- Quote from recent ISRO/IMD report on urban heat

Slide 3: Solution Overview

- System diagram showing: Satellite Data → AI Processing → Actionable Insights
- 3 key features: Real-time Detection, AI Prediction, Smart Recommendations
- Quick demo screenshot of your dashboard
- "Powered by freely available ISRO/NASA satellite data"

Slide 4: Data & Methodology

- Data sources table: Landsat, Sentinel, MODIS, WorldPop
- LST formula visual (simplified, not intimidating)
- Step-by-step flow: Raw DN → BT → LST → UHI → Hotspots
- Visual: before/after showing raw band vs processed LST

Slide 5: Hotspot Detection

- Side-by-side maps: city overview | zoomed hotspot zones
- Key finding: "18 critical zones identified affecting 2.3M residents"
- Top-5 hotspot table with severity scores
- Getis-Ord Gi* visualization (significance map)

Slide 6: AI/ML Approach

- Feature list: 8 input variables with visual icons
- Model architecture: Random Forest (200 trees, 8 features)
- Results: R^2=0.82, RMSE=1.2°C — with visual gauge
- SHAP feature importance chart

Slide 7: Live Dashboard

- Full-screen screenshot of Streamlit dashboard
- 4 labeled callouts pointing to key features
- "QR code" or URL if deployed on Streamlit Cloud
- Note: live demo follows

Slide 8: Key Results

- Summary statistics: cities analyzed, hotspots detected, population at risk
- Most significant finding (e.g., "industrial zones are 6.2°C hotter than parks")
- Validation: model vs ground truth comparison chart

Slide 9: Cooling Recommendations

- 5 intervention types with estimated temperature reductions
- Priority matrix: Impact vs Cost quadrant chart
- Example: "If top-3 interventions implemented → 47,000 heat illness cases prevented"
- Connects to Smart City Mission and National Action Plan on Climate Change

Slide 10: Impact & Scalability

- "This system works for any Indian city"
- Beneficiaries: NDMA, Smart City SPVs, Urban Local Bodies, IMD
- Social impact: vulnerable populations (elderly, outdoor workers, slum dwellers)
- Economic impact: reduced healthcare costs, improved worker productivity

Slide 11: Future Scope

- Real-time alerts: integrate with IMD weather API for heat stress warnings
- District-level deployment: scale to all 640 districts
- Deep learning: CNN on raw satellite imagery for end-to-end processing
- Mobile app: neighborhood-level cooling recommendations for citizens
- Integration with ISRO Bhuvan portal

Slide 12: Thank You

- GitHub repository QR code
- Live dashboard URL (if deployed)
- Team contact information
- "Questions? We'd love to show you the live demo."

15. MVP vs Advanced Features

15.1 MVP — Must Complete in 5 Days

✓ **Tip:** If you run out of time, these are the non-negotiables. Everything else is bonus.

Priority	Feature	Effort	Why Essential
P0	LST calculation for 1 city and 1 date	4 hours	Core output — without this nothing works
P0	UHI zone classification (5 classes)	1 hour	Differentiates your solution
P0	Top-10 hotspot detection (any method)	3 hours	The key deliverable judges will look for
P0	Basic Folium map in Streamlit	3 hours	Visual demo impact
P1	Random Forest model with basic features	5 hours	AI/ML requirement for hackathon
P1	Cooling recommendations (rule-based)	2 hours	Actionability is what wins hackathons
P1	Population overlay on hotspot map	2 hours	Shows social impact dimension
P1	Model metrics slide in PPT	1 hour	Demonstrates rigor to judges
P2	NDVI, NDBI, NDWI layers on map	3 hours	Strengthens methodology
P2	Download CSV of hotspot report	1 hour	Practical usability feature

15.2 Advanced Features — If Time Permits

■ **Note:** Implement these only after MVP is fully working and tested.

Feature	Effort	Technical Approach
Multi-city comparison dashboard	6 hours	Loop LST pipeline over 3-5 cities, add city dropdown
Temporal analysis (multi-date LST)	8 hours	Download 5+ dates, animate LST change on map
DBSCAN spatial clustering	3 hours	Replace grid-based hotspots with DBSCAN zones
Getis-Ord Gi* statistics	4 hours	pysal esda library — proper spatial statistics
SHAP feature importance	2 hours	pip install shap, TreeExplainer(rf).shap_values(X)
CNN on raw imagery (deep learning)	12 hours	PyTorch ResNet-50, requires GPU
Future scenario simulation	8 hours	Slider to change NDVI/NDBI, rerun ML prediction
Streamlit Cloud deployment	1 hour	streamlit share, GitHub push, auto-deploys

Feature	Effort	Technical Approach
PDF report auto-generation	4 hours	reportlab to generate city heat stress report PDF
Mobile-responsive PWA	10 hours	React.js frontend with Folium map embed
IMD API weather integration	6 hours	Fetch current temperature, humidity, heat index
Alert system (email/SMS)	5 hours	Threshold-based triggers, Twilio for SMS

15.3 Time Budget — Recommended Allocation

Day	Primary Focus	Hours	Buffer
Day 1	Setup + Data Acquisition	8h	1h for troubleshooting installs
Day 2	Preprocessing + LST Calculation	8h	1h for data quality issues
Day 3	Hotspot Detection + ML Model	9h	1h for model debugging
Day 4	Dashboard Development	8h	2h for UI polish
Day 5	Integration + Testing + PPT	8h	2h for rehearsal
Total	Full Project Execution	41h	7h contingency buffer

Final Advice from Your ISRO Mentor

- **Choose Delhi or Hyderabad as your city:** Best data availability, well-documented heat island patterns, lots of reference studies to cite.
- **Work on demo data in parallel:** While your Data Lead downloads real data, your Lead Dev should build the dashboard skeleton with synthetic data so no one is blocked.
- **$R^2 > 0.70$ is sufficient:** Don't over-engineer the ML model. A clean, well-explained Random Forest beats a messy deep learning model judges can't understand.
- **Make the map the hero:** Judges at a geospatial hackathon LOVE maps. Your Folium map should be the first thing they see. Make it beautiful with a good color scale.
- **Quantify your impact:** "Our system can help 2.3 million residents" beats "our system detects hotspots". Always tie technical results to human impact numbers.
- **Have a backup plan:** Pre-process your demo data and commit it to GitHub. If live processing fails during the demo, load the pre-processed version instantly.